4.2 Laser Scanners

One of the most widely used types of range/bearing sensors is the 2D LiDAR (*Light-based Detection And Ranging*) or laser scanner. Their principle of operation is simple: the sensor emits a pulse of light of a particular frequency, and waits for it to be reflected back by the environment. Once the reflected pulse is received, the distance it has traveled can be calculated from the time-of-flight (TOF).

A 2D scanner repeats this operation many times while rotating around its own center, thus obtaining information about the distance to the environment at different angles on its plane of operation. The main parameters that describe a laser scanner are:

- The angle increment between one measurement and the next is the **angular resolution** of the scanner, and determines how detailed the complete measurement is.
- The **field of view (FOV)** determines the angle limits of where the scan starts and stops. While some commercial LiDARs are capable of scanning all around themselves, it is often necessary to limit the FOV due to where the sensor is mounted on the robot (as we want to prevent the sensor from observing the body of the robot itself).
- The **max distance**, past which the sensor is not able to produce a measurement at all.
- The **noise**, of course. You know how this goes by now: sensor measurements are never fully reliable. Depending on the specifics of the sensor, the uncertainty of the measurements might only be reported for the *distance* reading (that is, a scalar **variance**), or for both the distance and the angle (a **covariance matrix**).

Top row, 2D laser scanners from SICK. Bottom row, left, 2D laser scanners from Hokuyo, right, illustration of the field of view of some models of this brand.

Let's create a simulated Laser Scanner with these parameters and test it out!

In [16]:
```
import numpy as np
import matplotlib.pyplot as plt
import ipywidgets

import sys
sys.path.append("..")

from utils.laser.laser2D import Laser2D
from utils.DrawRobot import DrawRobot
from utils.tcomp import tcomp
```

Creating the scanner and the map

We will use the `Laser2D` class imported above. The constructor of this class expects the parameters we just discussed:

- `FOV` -- Sensor field of view (radians)
- `resolution` -- Sensor resolution (radians)
- `max_distance` -- Max operating distance of the sensor (meters)
- `noise_cov` -- covariance matrix characterizing sensor noise
- `pose` -- sensor pose (column vector with x and y positions, and orientation theta)

The following function shows you how to create an instance of the `Laser2D` class:

In [17]:
```
def create_laser_at_origin(FOV, resolution, max_distance, noise_cov):
    origin_pose = np.vstack([0, 0, 0])
    laser = Laser2D(FOV, resolution, max_distance, noise_cov, origin_pose)
    return laser
```

The `Laser2D` class has two important methods: `set_pose()`, to tell it where to take an observation from; and `take_observation()`, which does exactly what you think. However, you might be asking "an observation of *what*, exactly? We haven't told the laser what the environment is like." Well, you're right, so let's do just that!

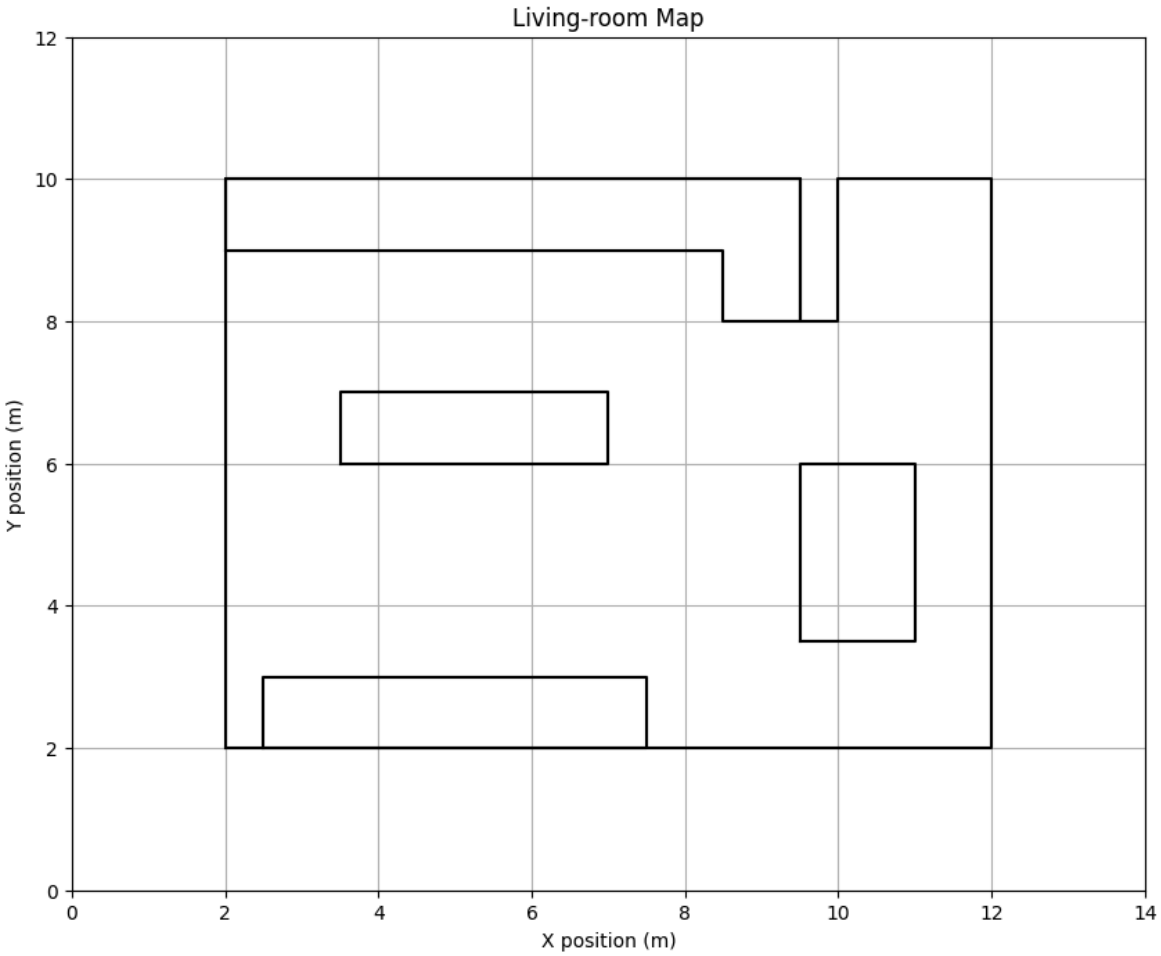
We will define the map as a list of polygonal **contours**. In the following cell, you will see an example:

In [18]:
```
def create_room_map():
    # Define the environment
    # A polygon contour is defined as two arrays, specifying the x and y coordin
    # The polygon is generated by connecting (x_t, y_t) to (x_t+1, y_t+1) with a

    # NaN denotes the end of each polygon
    walls = np.array([[2, 12, 12, 10, 10, 9.5, 9.5, 2, 2, np.nan],
                      [2, 2, 10, 10, 8, 8, 10, 10, 2, np.nan]])

    kitchen = np.array([[2, 8.5, 8.5, 9.5, 9.5, 2, 2, np.nan],
                        [9, 9, 8, 8, 10, 10, 9, np.nan]])

    sofa = np.array([[3.5, 7, 7, 3.5, 3.5, np.nan],
                     [6, 6, 7, 7, 6, np.nan]])

    tv_cabinet = np.array([[2.5, 7.5, 7.5, 2.5, 2.5, np.nan],
                           [2, 2, 3, 3, 2, np.nan]])

    table = np.array([[9.5, 11, 11, 9.5, 9.5, np.nan],
                      [3.5, 3.5, 6, 6, 3.5, np.nan]])


    virtual_map = np.concatenate([walls, kitchen, sofa, tv_cabinet, table], axis
    return virtual_map

virtual_map = create_room_map()
# Print map shape
print('Map shape:', virtual_map.shape)
```

```
Map shape: (2, 36)
```

The `plot_virtual_map()` function is provided to help you visually interpret the process and understand what is happening:

In [19]:
```
def plot_virtual_map(virtual_map, dimensions_only = False):
    """
    Plots the robot pose, virtual map, beam endpoints, and optionally grid cells

    Parameters:
    - virtual_map: environment representation using lines.
    """

    fig, ax = plt.subplots(figsize=(10, 8))

    # Plot the virtual map (environment boundaries or walls)
    if not dimensions_only:
        plt.plot(virtual_map[0, :], virtual_map[1, :], 'k-')

    # Set grid and axis limits
    plt.grid()
    plt.xlim([np.nanmin(virtual_map[0])-2,np.nanmax(virtual_map[0])+2]) # nanmin
    plt.ylim([np.nanmin(virtual_map[1])-2,np.nanmax(virtual_map[1])+2])

    # Title and axis labels
    plt.title('Living-room Map')
    plt.xlabel('X position (m)')
    plt.ylabel('Y position (m)')

    return fig, ax

plot_virtual_map(virtual_map);
```



ASSIGNMENT 1: Taking an observation

To use the scanner, we will call the `take_observation()` method, passing it the map as an argument.

Similarly to the landmark-based sensors we saw in the last notebook, our laser scanner will produce observations as angle-distance pairs. In other words, the measurements are in *polar coordinates*. Remember that you can express a sensor measurement given in polar coordinates ($z_p = [r, \alpha]^T$) as cartesian coordinates ($z_c = [z_x, z_y]^T$) with the following expression:

$$z_c = \begin{bmatrix} z_x \\ z_y \end{bmatrix} = \begin{bmatrix} r\ cos\alpha \\ r\ sin\alpha \end{bmatrix}$$

Remember also that all the observations are taken from the point of view of the sensor, and thus are expressed in its local system of reference. To express the measurements in world space (on the map), you can use our old trick: **pose composition**.

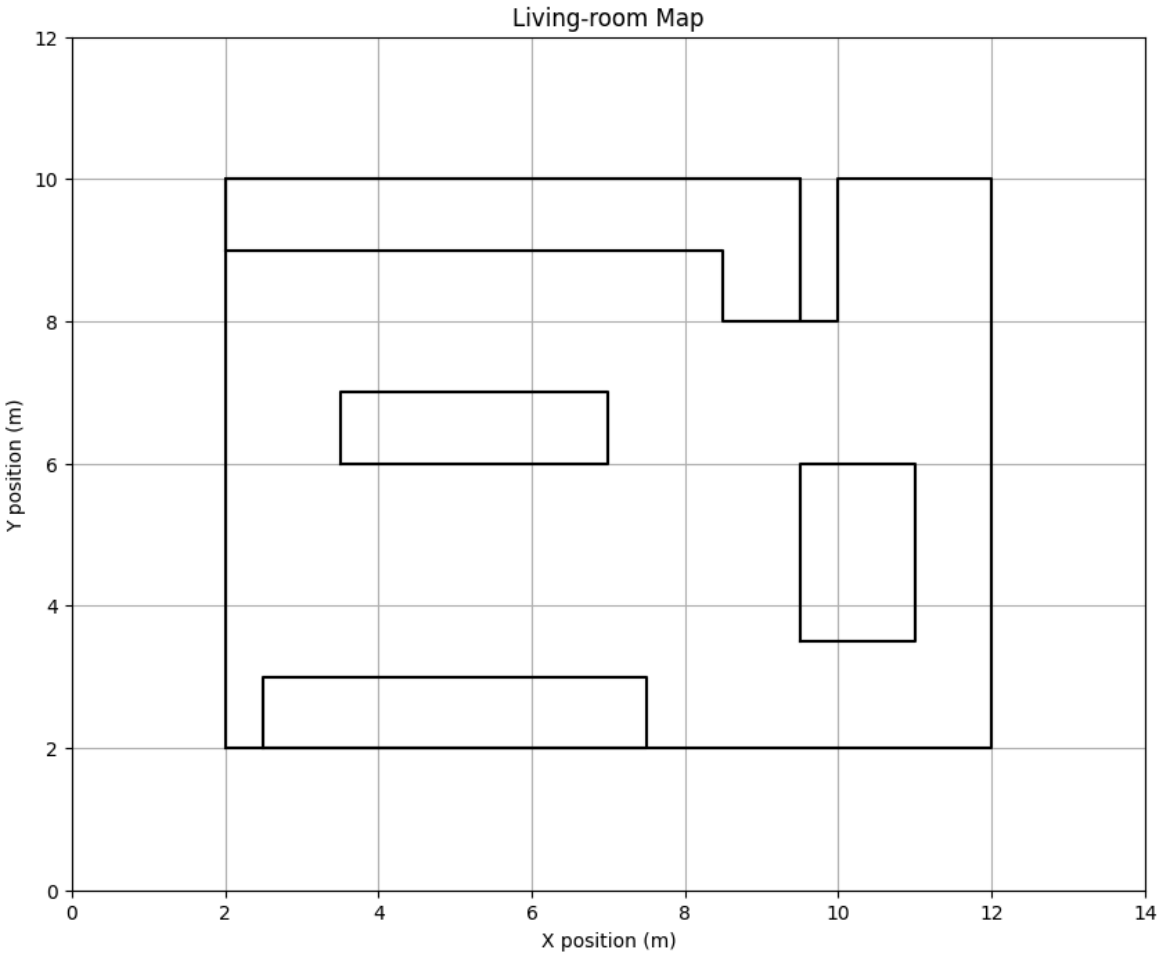
Let's put all this into practice. Implement a function which takes a sensor reading z (which is a **list** of points, in polar coordinates and relative to the sensor pose), and plots those points on the map as a *point cloud*.

In [20]:
```
def plot_observations(z, pose, fig, ax, draw_lines = True, point_color = 'r'):
    """Draw a sensor observation taken from a given pose on a figure.
```

```
    Keyword arguments:
    z -- Laser observation
    pose -- Pose from which the observation was taken
    fig, ax -- Figure to plot in
    """
    for i in range (z.shape[1]):
        distance = z[0,i]
        angle = z[1,i]

        # calculate the cartesian coordinates of the observed point in local spa
        x_local = distance * np.cos(angle)
        y_local = distance * np.sin(angle)

        # compose with the robot pose to express the point in global space
        pose_obs_global = tcomp(pose, np.vstack([x_local, y_local, 0]))
        x = pose_obs_global[0]
        y = pose_obs_global[1]

        if draw_lines:
            ax.plot([x, pose[0]], [y, pose[1]], 'g--', linewidth=1)
        ax.plot(x, y, 'o', markersize=4, color=point_color)
```

Use the following code to test it out:

In [21]:
```
def run_laser_and_plot(laser, pose, fig, ax, draw_lines = True, color='r'):
    laser.set_pose(pose)

    # take the measurement
    measurement = laser.take_observation(virtual_map)

    # Plot!
    DrawRobot(fig, ax, pose, axis_percent=0.015, color=color, linewidth=1.5);
    plot_observations(measurement, pose, fig, ax, draw_lines, color)
    return fig, ax


def test_laser(FOV, resolution, max_distance, distance_noise, angle_noise):
    laser = create_laser_at_origin(FOV, resolution, max_distance, np.diag([dista

    fig, ax = plot_virtual_map(virtual_map);
    run_laser_and_plot(laser, np.vstack([6, 4, np.pi/2]), fig, ax);

ipywidgets.interact(
    test_laser,
    FOV=ipywidgets.FloatSlider(3 * np.pi/2, min=0.2, max=2 * np.pi),
    resolution=ipywidgets.FloatSlider(0.2, min=0.02, max=0.5, step=0.01),
    max_distance=ipywidgets.FloatSlider(10, min=1, max=10),
    distance_noise=ipywidgets.FloatSlider(0.01, min=0.0, max=0.2, step=0.01),
    angle_noise=ipywidgets.FloatSlider(0.001, min=0, max=0.2, step=0.01)
)
```




```
interactive(children=(FloatSlider(value=4.71238898038469, description='FOV', max=
6.283185307179586, min=0.2), …
```

Out[21]: `<function __main__.test_laser(FOV, resolution, max_distance, distance_noise, an gle_noise)>`

Expected output:

Thinking about it

Having completed the code above, you will be able to **answer the following questions**:

- Which has a greater impact on the reliability of the measurements: the variance of the distance reading, or the variance of the angle at which the measurement happens? Which would you guess is larger in real sensors, and why?

  Orientation has almost no effect because a laser measurement depends only on the range and the angle of the beam, not on the absolute orientation of the landmark. Therefore, we ignore orientation and treat each landmark simply as a point in space.

- Without actually implementing it: how could we calculate the uncertainty about the position of each of the measured points (in cartesian coordinates)? *Hint: we already did this for the landmarks!*

To obtain the uncertainty of each measured point in Cartesian coordinates, we treat each laser beam (r,α) as a random variable with diagonal covariance (range variance and angle variance). Then we convert this uncertainty to Cartesian form using the Jacobian of the polar→Cartesian transformation, exactly as we did for landmarks: we multiply the Jacobian, the polar covariance, and the Jacobian transpose.

- We are assuming we know the pose of the robot, but what if there was also uncertainty associated with it? How do you combine both sources of uncertainty? *Hint: we **also** did this for the landmarks!*

  If the robot pose were also uncertain, we would combine both sources of uncertainty by propagating the robot-pose covariance through the pose–point composition, just as we did with landmarks. First, we would transform the beam's uncertainty from polar to Cartesian. Then we would add the contribution of the robot-pose covariance using the corresponding Jacobians of the composition

ASSIGNMENT 2: Exploring the environment

Now that we can plot the measurements in global coordinates, we should be able to roughly reconstruct the contours of the map by moving around the environment and repeatedly taking observations from multiple poses. Let's try that out:

In [ ]:
```
def explore_environment():
    laser = create_laser_at_origin(FOV = 3 * np.pi/2,
        resolution = 0.05,
        max_distance = 10,
        noise_cov = np.diag([0.03, 0.01]))

    # The robot must make these movements, in this order
    # 0, 0, 0
    # 2, 0, 0
    # 2, 0, 0
    # 0, 2, np.pi/2
    # 2, 0, 0
    # Format them correctly in the following list

    # each pose and its corresponding observations will have a different color
    pose_increments_list = [
        (np.vstack([0, 0, 0]),        'orangered'),
        (np.vstack([2, 0, 0]),        'olive'),
        (np.vstack([2, 0, 0]),        'teal'),
        (np.vstack([0, 2, np.pi/2]),  'magenta'),
        (np.vstack([2, 0, 0]),        'indigo'),
    ]

    fig, ax = plot_virtual_map(virtual_map, True); #create the base figure with

    current_pose = np.vstack([4,4,0]) # starting pose

    # take a step and read the sensor at the new pose
    for step, color in pose_increments_list:
        # Actualizar pose global tras cada desplazamiento relativo
        current_pose = tcomp(current_pose, step)

        # Tomar medida y dibujarla
```

```
        run_laser_and_plot(laser, current_pose, fig, ax, False, color)
explore_environment()
```

Expected output:

This looks pretty promising! Looking at this image, it is quite easy to get an intuition for the shape of the environment and the presence of obstacles. However, due to the noise in the sensor observations, it is difficult to tell where *exactly* the contours of the map really are.

This is the problem of **mapping**, which we will talk about in more detail in chapter 6. It probably won't surprise you to learn that, in order to deal with these noisy observations, we will tackle this problem probabilistically.

An even more complicated version of this process is necessary when the pose of the robot is uncertain, as we can't even reliably place the noisy points on the global map. To solve this (more realistic) case, we would need to simultaneously figure out the correct pose of the robot *and* build the map. This is the wonderfully named problem of **SLAM:** *Simultaneous Localization And Mapping*, which we will talk about in chapter 7.

AI Appendix

A.1 How I used AI

- **Model/tool:**

  ChatGPT (GPT-5)
- **What I asked AI to help with (bullets):**

  Clarify concepts about sensor uncertainty and landmark models.Explain covariance propagation using Jacobians. Refine and rewrite answers for the "Thinking about it" sections.Improve the clarity and structure of the memo.
- **Best prompt I used (paste):**

  "Explain clearly how range and angle variances propagate into Cartesian uncertainty and how robot-pose uncertainty combines with sensor noise, using the same reasoning as in the landmark exercises."
- **What I kept vs. changed from the AI output:**

  I kept the main ideas and equations, but rewrote the explanations to be more concise and better aligned with the style of the lab. I also adjusted the level of detail so it matched the expectations of the notebook.

A.2 AI review

Ask one generative AI to review your memo using this prompt:

```
Act as a technical reviewer of this computer vision lab
notebook. Read my memo and return exactly 5 bullet points:
```

```
- two on theory (key ideas),
- two on practice (relevant programming concepts of python used,
not particular functions or variables),
- one on how to improve the code (most critical aspect:
limitations, parameter choices, runtime/robustness, possible
bugs).

Keep each bullet ≤15 words and make them specific to my work.
```

Paste its **5** items here:

- **(Theory 1)** AI claim:

  The memo clearly explains how polar noise transforms into Cartesian uncertainty.

- **(Theory 2)** AI claim:

  You correctly describe how robot-pose uncertainty combines with sensor noise.

- **(Practice 1)** AI claim:

  Good use of Jacobians for transforming uncertainties across coordinate frames.

- **(Practice 2)** AI claim:

  Consistent use of matrix operations to propagate covariances.

- **(Improvement)** AI claim:

  Add checks for singular or ill-conditioned Jacobians to prevent numerical issues.

Finally, **do you agree with them?**, **would you have selected others?**

Yes, I agree: the points highlight the key theoretical and practical aspects of my work. I might also add a remark about validating the sensor parameters, but overall the feedback is appropriate and consistent with the notebook.

In [ ]: